

1 Lesson 30-09-26

1.1 Memory

Last week we went over the structure of an embedded system MCU. Now we focus on memory, with the target of defining a memory map and defining a linker script for it.

1.1.1 Types of memory

Let's quickly go over the *types* of memory available to us.

SRAM

- **SRAM** (static RAM) sits on the CPU's memory bus: every byte has an address, and the CPU reads or writes it with a normal load/store;
- **Fast**: on a microcontroller, one access takes about one clock cycle;
- **Byte-writable**: we can change a single byte, any time, as often as we like;
- **Volatile**: power off and everything is lost; at power-on the content is **undefined** (random bits, not zeros).

SRAM is where anything that **changes** while the program runs must live: variables, the stack.

NOR flash

- Also sits on the memory bus, like a ROM: every word has an address, random-access reads are fast;
- So the CPU can **fetch** instructions directly from it (common in MCUs): **execute-in-place** (XIP). No copy needed;
- **Non-volatile**: the program is still there after power-off;
- Writing is **not** a store instruction:
 - First **erase** a whole sector (e.g. a few KB) → all bits become 1;
 - Then **program** words (bits can only go 1 → 0)
 - Microseconds to milliseconds, through a special command sequence
- Result: at run time, NOR flash is effectively **read-only**.

NAND flash and disks

- NAND flash (SD cards, SSDs) and hard disks are **not** on the memory bus: they are reached through a controller, one **block** (page) at a time, e.g. 512 B – 4 KB;
- A block has no CPU address → the CPU **cannot fetch instructions** from it, nor read a single variable;
- Contents must first be **copied into RAM**, then used from there;

This is what a desktop PC does: the program sits on the SSD, and the OS **loader** copies it into RAM before running it. A microcontroller has no loader: its program already sits in NOR flash, at the address it runs from.

1.1.2 Effect on program sections

The different memory types we have available play a role in how we define our *program headers* (or *segment headers*) for embedded programs.

Technology	On the bus	Executable	Writable at runtime	Persistent
SRAM	Yes	Yes (XIP)	Yes, bytes	No
NOR flash	Yes	Yes (XIP)	No (erase/program)	Yes
NAND / disk	No (blocks)	No	Via a driver	Yes

The CPU sees one 4 GB address space. On every Cortex-M the **regions** are fixed by the architecture; each chip decides what it populates.

Address range	Region	On mps2-an385
0x0000_0000–0x1FFF_FFFF	Code	4 MB at 0x0: the program ("FLASH")
0x2000_0000–0x3FFF_FFFF	SRAM	4 MB at 0x2000_0000: "RAM"
0x4000_0000–0x5FFF_FFFF	Peripherals	UART, timers...
0x6000_0000–0xDFFF_FFFF	External RAM / devices	Not used here
0xE000_0000–0xE00F_FFFF	CPU's private registers	Control of the CPU itself (at 0xE000_E000)

Tabella 1: Cortex-M memory map and the regions populated by mps2-an385.

1.1.3 Linker script

This leaves us with the following linker script, which we can find very useful to analyze:

```

1  /* Linker script for mps2-an385 machine (ARM Cortex-M3, MPS2 platform). */
2
3  /* Physical memory regions of the board. */
4  MEMORY
5  {
6      /* code, rodata (XIP) */
7      FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 4M
8
9      /* data, bss, stack */
10     RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
11 }
12
13 /* Entry point, also entry 1 of vector table (reset handler). */
14 ENTRY(Reset_Handler)
15
16 /* initial SP = top of RAM; the stack grows downward from here */
17 _estack = ORIGIN(RAM) + LENGTH(RAM);
18
19 /* 4 KB reserved for the stack*/
20 _Min_Stack_Size = 0x1000;
21
22 /* combine input (ELF) sections into output sections */
23 SECTIONS
24 {
25     /* Vector table first, at address 0, so the CPU finds the initial SP
26      * and reset vector where the architecture requires them. */
27     .isr_vector :

```

```

28 {
29     . = ALIGN(4);                /* vector table must be word-aligned */
30     KEEP(*(.isr_vector))        /* KEEP: never garbage-collect it */
31     . = ALIGN(4);                /* keep alignment */
32 } > FLASH                       /* place in FLASH */
33
34 /* Executable code and read-only data. */
35 .text :
36 {
37     . = ALIGN(4);                /* word-align the start of this section */
38     *(.text*)                   /* all code from every input object */
39     *(.rodata*)                 /* string literals, const tables */
40     . = ALIGN(4);                /* word-align the end so (_etext) */
41     _etext = .;                 /* end-of-text marker */
42 } > FLASH                       /* place in FLASH */
43
44 /* FLASH address where .data's initial image is stored: the technique
45 allows for reserving space in RAM while storing in flash. */
46 _sidata = LOADADDR(.data);
47
48 /* Initialized variables: they live (run) in RAM but their initial
49 * contents are stored in FLASH and copied over by Reset_Handler. */
50 .data :
51 {
52     . = ALIGN(4);                /* word-align the start of .data */
53     _sidata = .;                /* start of .data in RAM */
54     *(.data*)                   /* every initialized-variable input
55 section */
56     . = ALIGN(4);                /* word-align the end */
57     _edata = .;                /* end of .data in RAM - copy loop stops
58 here */
59 } > RAM AT > FLASH             /* place in FLASH, access in RAM */
60
61 /* Zero-initialized variables: reserve RAM, no stored image. */
62 .bss :
63 {
64     . = ALIGN(4);                /* word-align the start of .bss */
65     _sbss = .;                  /* start of .bss */
66     *(.bss*)                   /* every zero-initialized input section */
67     *(COMMON)                  /* legacy tentative-definition symbols */
68     . = ALIGN(4);                /* word-align the end */
69     _ebss = .;                  /* end of .bss - zero loop stops here */
70 } > RAM

```

For now we intentionally skip the *heap* and other advanced features, but this should be pretty useful to understand how the memory is laid out. Let us remember that here we are defining memory **regions**, declaring memory **sections**, and expecting the linker to generate for us memory **segments** (program headers) which place the data in our segments within more "blocks" (that is, segments) to load in different memory regions (the ones seen above).

1.1.4 LMA vs VMA

Let us also note the *LMA/VMA* distinction:

- **LMA** (*Load Memory Address*) is where the initial bytes of data are located in the executable/image (the program headers which we load);

- **VMA** (*Virtual/Run-time Memory Address*) is where the data in the section expects to live when the program executes.

The gimmick at lines 45-56 allows us to set a LMA for the data section inside the flash memory, and set its VMA to the address it will be copied to in RAM. Of course, the task of copying the data itself is assigned to the reset handler.

We also note that the `.bss` initialize sector has a VMA (where to access uninitialized data) but no LMA (nothing to load, the reset handler just zeroes out).

1.1.5 Vector table

One important region of the memory (more specifically, FLASH memory) is the **vector table**.

- It is a **table** of function addresses at a fixed place (address 0x0 at reset).
- Each **exception** (an event that makes the CPU stop and run a specific handler: reset, a fault, ...) has a numbered **slot**; the CPU reads slot *N* to find where to jump,
- Slot 0 is special: it contains not an address of code, but the initial stack pointer (SP), which gets loaded automatically by the CPU. This means that the reset handler is at index 1 (not 0).

In `boot/startup.s` (base), section `.isr_vector`:

```
1 isr_vector:
2   .word _estack          /* 0: initial SP   (C: &_estack)    */
3   .word Reset_Handler   /* 1: reset       (C: Reset_Handler) */
```

The stored value of word 1 is `Reset_Handler` | 1: the low bit marks Thumb code. Nothing ever *calls* `isr_vector`; only the hardware reads it.

1.2 Registers

We now go over the ARM architecture programmer registers. In all ARM processors, the following registers are available and accessible in any processor mode:

- 13 general-purpose registers: R0-R12: user variables, with call arguments passed through R0-R3;
- One *Stack Pointer* (SP): address of the top of the stack;
- One *Link Register* (LR): where to return after a function call;
- One *Program Counter* (PC): address of the next instruction to execute;
- One *Status Register* (xPSR): flags, Thumb bit, current exception.

Some things to note at this level which may not be clear:

- There are no “variables” at this level: only these registers and memory.
- The stack grows **downwards**: a push first does `SP -= 4`, then writes.

xPSR packs three things into one register:

The CPU is always in one of two **modes**:

Bits	Content
31–28	flags N Z C V (negative, zero, carry, overflow), set by <code>cmp</code>
24	T : Thumb state — must always be 1 on Cortex-M
8–0	number of the exception being handled (0 = none)

Tabella 2: xPSR fields

- **Thread mode** — normal code: `Reset_Handler`, `main()` (xPSR bits 8–0 = 0).
- **Handler mode** — running an exception handler (bits 8–0 = its number).

Code can also be *privileged* or not; everything in this unit runs privileged.

1.3 ISA

ARM Cortex-M uses a **load/store** architecture: arithmetic and logical operations operate on registers, while memory is accessed explicitly through load and store instructions.

- `ldr`: **load** data from memory into a register;
- `str`: **store** data from a register into memory;
- Arithmetic instructions do not directly operate on memory operands.

Let's have a table of example memory operations:

Assembly	C equivalent
<code>ldr r3, [r0]</code>	<code>r3 = *r0;</code>
<code>str r3, [r1]</code>	<code>*r1 = r3;</code>
<code>ldr r3, [r0], #4</code>	<code>r3 = *r0; r0 += 4;</code> <code>// i.e. r3 = *src++;</code>
<code>str r3, [r1], #4</code>	<code>*r1 = r3; r1 += 4;</code> <code>// i.e. *dst++ = r3;</code>
<code>strb r1, [r0, #7]</code>	<code>((uint8_t *)r0)[7] = r1;</code> <code>// one byte, at offset 7</code>

1.4 Post- and pre-increment

In ARM we have **pre-indexed** and **post-indexed** memory operations, that is control over the time when the base register is updated.

Being more specific, ARM allows load and store instructions to optionally *alter* the address contained within a base register. 3 forms are available:

- `ldr r0, [r1, #4]` is equivalent to immediate offset addressing in RISC-V, with `#4` being the immediate operand;
- `ldr r0, [r1, #4]!`, with the additional `!`, means performing a *write-back* or **pre-indexing** operation. The calculated address, obtained by adding the offset, is *written back* into the base register before the load operation. The result we get is that of a load from the base address plus the offset, with the base address reflecting the change after execution. Compared to RISC-V, this saves an instruction;

- `ldr r0, [r1], #4` means performing a **post-indexing** operation. This means increasing the value of the base register by offset, after performing the load. We can think of this as being analogous to the `++` syntax available in C. This also saves an instruction compared to RISC-V.

As we can see, the main point of this complication is to save a few instructions when accessing arrays, or generally where striding memory access patterns are needed. Since a word is 4 bytes on this machine, this is useful for walking through an array or copying a block of memory.

For example:

```
1 copy:
2     ldr r3, [r0], #4
3     str r3, [r1], #4
4     b   copy
```

copies one word at a time while automatically advancing both pointers.

1.5 Loading constants and addresses

Assembly code also needs mechanisms for loading constants, referring to addresses, defining labels, and changing the control flow.

Let's have a table of example (pseudo and normal) instructions for constant loading.

Assembly	C equivalent
<code>ldr r0, =_estack</code>	<code>r0 = (uint32_t)&_estack;</code>
<code>mov sp, r0</code>	<code>sp = r0;</code>
<code>movs r3, #0</code>	<code>r3 = 0;</code>
<code>copy_data:</code>	label: a name for this address
<code>b copy_data</code>	<code>goto copy_data;</code>
<code>bl main</code>	<code>main();</code>

The pseudo-instruction

```
1 ldr r0, =_estack
```

loads the value represented by `_estack` into `r0`. This is often an address, but the syntax can be used for arbitrary 32-bit constants.

A large constant generally cannot be encoded directly in a single Thumb instruction. The assembler can therefore place the constant in a nearby **literal pool** and generate a PC-relative load.

For example, GDB may show:

```
1 ldr r0, [pc, #92]
```

The directive

```
1 .ltorg
```

asks the assembler to emit the current literal pool at that point.

Note that a label names an address in the program:

```
1 copy_data:
2     ...
```

A branch transfers execution to that address:

```
1 b copy_data
```

This is analogous to a C `goto`, although assembly labels are addresses in the instruction stream rather than variables.

The instruction `bl` means **branch with link**. It branches to the target and simultaneously saves the return address in the **link register (LR)**. This is the normal mechanism used for a subroutine call:

```
1 bl main
```

A corresponding return normally uses the address stored in `lr`.

1.5.1 Branches

Assembly	C equivalent
<code>cmp r1, r2</code>	compute <code>r1 - r2</code> and update the flags
<code>bhs done</code>	if <code>(r1 >= r2)</code> goto <code>done</code> ; // unsigned comparison
<code>orr r1, r1, #0x10</code>	<code>r1 = 0x10</code> ;
<code>cpsid i</code>	disable interrupt acceptance
<code>cpsie i</code>	enable interrupt acceptance

Tabella 3: Conditional branches, bit operations and interrupt control.

`cmp` performs a subtraction internally (`r1 - r2`) but does not store the result. Instead, it updates the condition flags (N, Z, C, and V) in the xPSR.

A subsequent conditional branch examines these flags.

For example, `bhs` means “higher or same” for an **unsigned** comparison:

```
1 cmp r1, r2
2 bhs done
```

This corresponds to:

```
1 if (r1 >= r2)
2     goto done;
```

A C while loop therefore naturally becomes a test at the top, followed by a backwards branch at the bottom:

```
1 loop:
2     cmp r1, r2          @ while (r1 < r2) {
3     bhs done
4     ...                @     body
5     b    loop          @ }
6 done:
```

1.5.2 Interrupt masking

On Cortex-M, `cpsid i` disables the acceptance of maskable interrupts, while `cpsie i` enables them again:

```
1 cpsid i
2 ...
3 cpsie i
```

These correspond conceptually to the CMSIS functions `__disable_irq()` and `__enable_irq()`.

1.5.3 Assembler directives

Assembly source contains two fundamentally different kinds of lines:

1. **Instructions**, which become CPU instructions.
2. **Assembler directives**, which control how the assembler constructs the object file.

Directives generally begin with a period (.) and do not themselves execute on the CPU.

For example:

```

1 .section .isr_vector,"a",%progbits
2 .global isr_vector
3
4 isr_vector:
5     .word _estack
6     .word Reset_Handler
7
8 .section .text,"ax",%progbits
9 .global Reset_Handler
10 .thumb_func
11
12 Reset_Handler:

```

Here:

- `.section` selects or creates an ELF section.
- `.global` makes a symbol visible to the linker and other object files.
- `.word` emits a 32-bit value into the current section.
- `.thumb_func` tells the assembler that the following function is Thumb code.

Thus, `isr_vector` is not a function being executed by the CPU: the assembler is simply placing two 32-bit values into the `.isr_vector` section.

The corresponding idea in C is:

```

1 __attribute__((section(".isr_vector")))
2 const uint32_t isr_vector[] = {
3     (uint32_t)&_estack,
4     (uint32_t)Reset_Handler
5 };

```

Without `.global`, an assembly symbol is local to the object file, similar in effect to a static C symbol.

1.5.4 Thumb function addresses

Cortex-M executes Thumb instructions exclusively. Function pointers and exception-vector entries therefore have bit 0 set to indicate Thumb state.

For a symbol such as `Reset_Handler`, the assembler/linker can produce an address with bit 0 set when it is declared as a Thumb function:

```

1 .thumb_func
2 Reset_Handler:

```

For example, if the actual instruction address is `0x00000040`, the Thumb function pointer is represented as `0x00000041`.

The CPU uses this bit to identify the target as Thumb code; the actual instruction address is still aligned to an even address.

1.6 Faults

Let's look in more detail at **faults**, so that we can understand the whole of the *handlers* defined in the ISR vector table.

An **exception** is any event that interrupts the current code and causes the CPU to execute a handler whose address is stored in the vector table, switching to Handler mode.

#	Exception	Raised by	#	Exception	Raised by
0	<i>None</i>	Slot holds the initial SP	7–10	<i>Reserved</i>	Slot must hold 0
1	Reset	Power-on, reset button	11	SVCall	The svc instruction
2	NMI	Non-maskable interrupt	12	DebugMonitor	Debug events (unused here)
3	HardFault	Faults that cannot be handled elsewhere	13	<i>Reserved</i>	Slot must hold 0
4	MemManage	Memory protection violations	14	PendSV	A software request
5	BusFault	Bus access errors	15	SysTick	Built-in timer
6	UsageFault	Invalid instruction use, division by zero, ...	16+	IRQs	Device interrupts

At reset, only HardFault is active among the configurable fault handlers: faults that cannot be handled by their corresponding handlers escalate to HardFault. This corresponds to “escalate 3” behavior observed during debugging.

1.6.1 Handlers

The initial vector table contains only two words, but the Cortex-M processor expects one word per exception vector, for a total of 16 entries in the initial table. Slots 7–10 and 13 are reserved and must contain zero. Every other slot must contain a valid handler address, even if no specific handler has been implemented yet.

A default handler can be defined as follows:

```

1  .word HardFault_Handler      @ slot 3, and so on up to 15
2
3  .weak      HardFault_Handler
4  .thumb_set HardFault_Handler, Default_Handler
5
6 Default_Handler:
7  b Default_Handler           @ for (;;) {}

```

The equivalent declaration in C is:

```

1 void HardFault_Handler(void)
2 __attribute__((weak, alias("Default_Handler")));

```

A **weak symbol** provides a default implementation that can be overridden by a normal definition in another object file. If `main.c` defines its own `HardFault_Handler`, the linker uses that implementation instead of the default.

The completed vector table occupies 16 words:

$$16 \times 4 = 64 \text{ bytes} = 0x40.$$

Consequently, if the table starts at address 0x0, the `Reset_Handler` entry point moves from 0x8 to 0x40. Since Cortex-M executes Thumb instructions, its vector entry contains 0x41, with bit 0 set to indicate Thumb state.

1.7 CPU registers

One interesting thing about ARM is that some CPU settings live in *registers* which are stored at certain **memory addresses**.

For instance, the **SCB** (*System Control Block*) is a set of 32-bit registers inside the CPU, reached like memory in the system area at 0xE000E000 – 0xE000EFFF.

Let's recall the default processor registers:

Register	Purpose
r0–r12	General-purpose registers
r13 (SP)	Stack pointer
r14 (LR)	Link register; return address
r15 (PC)	Program counter
xPSR	Flags, execution state and exception number
MSP	Main stack pointer
PSP	Process stack pointer
PRIMASK	Masks configurable interrupts
BASEPRI	Masks interrupts below a priority threshold
FAULTMASK	Masks exceptions except NMI
CONTROL	Selects stack and privilege state in Thread mode

The exact register set depends on the Cortex-M implementation. In particular, `BASEPRI` is not implemented on every Cortex-M core.

The `SP` register refers to the currently selected stack pointer, either `MSP` or `PSP`. The processor uses the main stack pointer in Handler mode.

These peripherals expose registers through fixed memory addresses. They are accessed using ordinary load and store instructions.

Let's look at the main peripherals we have:

Block	Base address	Purpose
SCnSCB	0xE000E000	System control and feature registers
SysTick	0xE000E010	24-bit system timer
NVIC	0xE000E100	Interrupt enable, pending state and priorities
SCB	0xE000ED00	Exceptions, faults and processor configuration
MPU	0xE000ED90	Memory protection (optional)
DWT	0xE0001000	Watchpoints, cycle counter and profiling
FPB	0xE0002000	Flash patch and breakpoint unit
ITM	0xE0000000	Instrumentation and debug output
CoreDebug	0xE000EDF0	Debug control and status
TPIU	0xE0040000	Trace output interface

These addresses correspond to the Cortex-M3 memory map. Optional blocks may be absent or incompletely implemented in a particular processor or emulator.

The *System Control Block (SCB)* provides registers for processor configuration, exception handling and fault diagnosis.

Let's look at some important elements of the SCB:

Register	Offset	Purpose
CPUID	0x00	Processor identification
ICSR	0x04	Exception and interrupt state
VTOR	0x08	Vector table base address
AIRCR	0x0C	System reset and priority configuration
SCR	0x10	Sleep behavior
CCR	0x14	Processor configuration
SHPR1-3	0x18-0x23	System exception priorities
SHCSR	0x24	System handler enable and status
CFSR	0x28	Configurable fault status
HFSR	0x2C	HardFault status
DFSR	0x30	Debug fault status
MMFAR	0x34	Memory-management fault address
BFAR	0x38	Bus fault address

2 Real chips

QEMU's CPU is ready as soon as it leaves reset. A real MCU's startup code usually does more, often in a vendor function `SystemInit()`:

- **Clock / PLL:** the chip starts on a slow internal oscillator; the PLL multiplies the clock frequency up to the desired operating frequency.
- **Flash wait states:** Flash memory is slower than a fast CPU. The Flash controller must be configured to insert additional wait cycles *before* increasing the CPU clock frequency.
- **Watchdog:** a timer that resets the chip unless software periodically "feeds" it. On some chips the watchdog starts running immediately after reset, so the startup code must feed or disable it.
- **FPU (Cortex-M4F):** the floating-point unit is disabled at reset. It can be enabled through the CPACR register at 0xE000ED88; otherwise executing an FPU instruction can cause a UsageFault.
- **C++:** before entering `main()`, the startup code must call the static constructors stored in `.init_array`.